CSP-J 2022 (Junior)

2022-10-29, 8:30–12:00 (3.5 hours)

Problem 1: Exponentiation (乘方)

Time limit: 500 ms. Memory limit: 128.00 MB.

Background

To reproduce the scoring conditions during the contest, the time limit for this problem has been reduced by 50%.

Problem description

Xiaowen has just started taking part in informatics competitions. One day, she encounters the following problem: given positive integers a and b, find the value of a^b.

a^b is the product of b copies of a; for example, 2^3 is the product of three copies of 2, giving $2 \times 2 \times 2 = 8$.

"Easy!" Xiaowen thinks, and quickly writes a program, but it produces an error during testing.

Xiaowen soon realizes that all the variables in her program have type `int`. On most machines, the largest number that `int` can represent is $2^{31} - 1$, so her program will produce an error whenever the result of the calculation exceeds this number.

Since Xiaowen has only just learned to program, she is worried that calculating with `int` may cause problems. She therefore wants you to output `-1` as a warning if the value of a^b exceeds 10^9, and otherwise output the correct value of a^b.

However, Xiaowen still does not know how to implement this program, so she would like your help.

Input

The input consists of one line containing two positive integers a, b.

Output

Output one line: if the value of a^b does not exceed 10^9, output the value of a^b; otherwise, output `-1`.

Sample 1 input

```
10 9
```

Sample 1 output

```
1000000000
```

Sample 2 input

```
23333 66666
```

Sample 2 output

```
-1
```

Constraints

For 10% of the data, it is guaranteed that $b = 1$.
For 30% of the data, it is guaranteed that $b \leq 2$.
For 60% of the data, it is guaranteed that $b \leq 30$ and $a^b \leq 10^{18}$.
For 100% of the data, it is guaranteed that $1 \leq a, b \leq 10^9$.
Update 2022.11.14/2025.04.02: one additional hack test case was added on each of these dates.

Downloadable attachment

The problem page provides the archive `pow.zip`.

Problem 2: Decryption (解密)

Time limit: 1.00 s. Memory limit: 512.00 MB.

Problem description

Given a positive integer k, there are k queries, each giving three positive integers n_i, e_i, d_i and asking for two positive integers p_i, q_i such that

$$n_i = p_i \times q_i, \qquad e_i \times d_i = (p_i - 1)(q_i - 1) + 1.$$

Input

The first line contains a positive integer k, indicating that there are k queries.

The next k lines contain the queries; the i-th line contains three positive integers n_i, d_i, e_i.

Output

Output k lines, each containing two positive integers p_i, q_i representing the answer.

To make the output consistent, you must ensure that $p_i \leq q_i$.

If there is no solution, output `NO`.

Sample 1 input

```
10
770 77 5
633 1 211
545 1 499
683 3 227
858 3 257
723 37 13
572 26 11
867 17 17
829 3 263
528 4 109
```

Sample 1 output

```
2 385
NO
NO
NO
11 78
3 241
2 286
NO
NO
6 88
```

Sample 2 input

Provided as the downloadable file `decode/decode2.in` in the attachment `decode.zip`.

Sample 2 output

Provided as the downloadable file `decode/decode2.ans` in the attachment `decode.zip`.

Sample 3 input

Provided as the downloadable file `decode/decode3.in` in the attachment `decode.zip`.

Sample 3 output

Provided as the downloadable file `decode/decode3.ans` in the attachment `decode.zip`.

Sample 4 input

Provided as the downloadable file `decode/decode4.in` in the attachment `decode.zip`.

Sample 4 output

Provided as the downloadable file `decode/decode4.ans` in the attachment `decode.zip`.

Constraints

In the following, let $m = n - e \times d + 2$.

For 100% of the data, it is guaranteed that $1 \le k \le 10^5$ and, for every $1 \le i \le k$,

$$1 \le n_i \le 10^{18}, \qquad 1 \le e_i \times d_i \le 10^{18}, \qquad 1 \le m \le 10^9.$$

Test case	$k \le$	$n \le$	$m \le$	Special property
1	10^3	10^3	10^3	A solution is guaranteed
2	10^3	10^3	10^3	None
3	10^3	10^9	6×10^4	A solution is guaranteed
4	10^3	10^9	6×10^4	None
5	10^3	10^9	10^9	A solution is guaranteed
6	10^3	10^9	10^9	None
7	10^5	10^{18}	10^9	If a solution exists, $p = q$
8	10^5	10^{18}	10^9	A solution is guaranteed
9	10^5	10^{18}	10^9	None
10	10^5	10^{18}	10^9	None

Problem 3: Logical Expressions (逻辑表达式)

Time limit: 1.00 s. Memory limit: 512.00 MB.

Problem description

Logical expressions are an important concept and tool in computer science, involving logical values, logical operations, and the precedence of logical operations.

In a logical expression, an element can have only two possible values: 0 (representing false) and 1 (representing true). There are various possible logical operations between elements, but in this problem you need to consider only two: AND (denoted by `&`) and OR (denoted by `|`). Their rules are as follows:

$$0 \,\&\, 0 = 0 \,\&\, 1 = 1 \,\&\, 0 = 0, \qquad 1 \,\&\, 1 = 1;$$

$$0 \mid 0 = 0, \qquad 0 \mid 1 = 1 \mid 0 = 1 \mid 1 = 1.$$

A logical expression may also contain parentheses. During evaluation, the part inside parentheses is evaluated first; when the two types of operations occur at the same level, `&` takes precedence over `|`; operations of the same type at the same level are evaluated from left to right.

For example, the evaluation order of `0|1&0` is the same as that of `0|(1&0)`; the evaluation order of `0&1&0|1` is the same as that of `((0&1)&0)|1`.

In addition, some compilers for languages such as C++ use a strategy called "short-circuiting" to evaluate logical expressions: in an expression of the form `a&b`, the value of part `a` is evaluated first; if $a = 0$, the value of the entire logical expression must be 0, so there is no need to evaluate part `b`; similarly, in an expression of the form `a|b`, the value of part `a` is evaluated first; if $a = 1$, the value of the entire logical expression must be 1, so there is no need to evaluate part `b`.

You are given a logical expression and must compute its value and count how many times each of the two types of short-circuiting occurs during its evaluation. Note that a short circuit is not counted if it is contained within a part skipped by an outer short circuit: for example, in `1|(0&1)`, although `0&1` is a short circuit, the outer expression `1|(0&1)` itself short-circuits, so the value of `0&1` need not be computed, and this `0&1` must not be counted as a short circuit.

Input

The input consists of one line containing a nonempty string s representing the logical expression to evaluate.

Output

Output two lines: the first line contains a character `0` or `1`, representing the value of the logical expression; the second line contains two nonnegative integers, giving respectively the number of short circuits of the forms `a&b` and `a|b` that occur while evaluating the expression.

Sample 1 input

```
0&(1|0)|(1|1|1&0)
```

Sample 1 output

```
1
1 2
```

Sample 2 input

```
(0|1&0|1|1|(1|1))&(0&1&(1|0)|0|1|0)&0
```

Sample 2 output

```
0
2 3
```

Explanation of sample 1

The evaluation of this logical expression proceeds as follows; the comment on each line explains the operation on the preceding line:

```
0&(1|0)|(1|1|1&0)
=(0&(1|0))|((1|1)|(1&0)) // Parentheses show the evaluation order.
=0|((1|1)|(1&0))   // Evaluate the leftmost &: an a&b short circuit.
=0|(1|(1&0))       // Evaluate the middle |: an a|b short circuit.
=0|1               // Evaluate the middle |: an a|b short circuit.
=1
```

Sample 3 input

Provided as the downloadable file `expr/expr3.in` in the attachment `expr.zip`.

Sample 3 output

Provided as the downloadable file `expr/expr3.ans` in the attachment `expr.zip`.

Sample 4 input

Provided as the downloadable file `expr/expr4.in` in the attachment `expr.zip`.

Sample 4 output

Provided as the downloadable file `expr/expr4.ans` in the attachment `expr.zip`.

Constraints

Let $|s|$ denote the length of the string s.

For all data, $1 \leq |s| \leq 10^6$. It is guaranteed that s contains only the characters `0`, `1`, `&`, `|`, `(`, and `)`, and is a well-formed logical expression. It is guaranteed that there are no extra spaces at the beginning, in the middle, or at the end of the input string. It is guaranteed that s contains no redundant nested parentheses (that is, no substring of the form `((a))`, where `a` is a well-formed logical expression).

Test case	$\|s\| \leq$	Special condition
$1 \sim 2$	3	None
$3 \sim 4$	5	None
5	2000	1
6	2000	2
7	2000	3
$8 \sim 10$	2000	None
$11 \sim 12$	10^6	1
$13 \sim 14$	10^6	2
$15 \sim 17$	10^6	3
$18 \sim 20$	10^6	None

Here the special properties are as follows:

- Special property 1: s is guaranteed not to contain `&`.
- Special property 2: s is guaranteed not to contain `|`.
- Special property 3: s is guaranteed not to contain `(` or `)`.

Note

The following is a formal definition of a "well-formed logical expression":

- The strings `0` and `1` are well-formed.
- If the string `s` is well-formed, and `s` is not a string of the form `(t)` (where `t` is well-formed), then the string `(s)` is also well-formed.
- If the strings `a` and `b` are both well-formed, then the strings `a&b` and `a|b` are both well-formed.
- Every well-formed logical expression can be generated using the methods above.

Problem 4: Increasing Point Sequence (上升点列)

Time limit: 1.00 s. Memory limit: 512.00 MB.

Problem description

On a two-dimensional plane, you are given n integer-coordinate points (x_i, y_i), and you may freely add k more integer-coordinate points.

After freely adding k points, you must select some of the $n + k$ points and arrange them into a sequence such that the Euclidean distance between every two consecutive points is exactly 1, and both their horizontal and vertical coordinates are monotonically nondecreasing; that is,

$$x_{i+1} - x_i = 1, \quad y_{i+1} = y_i,$$

or

$$y_{i+1} - y_i = 1, \quad x_{i+1} = x_i.$$

Determine the maximum length of a sequence satisfying these conditions.

Input

The first line contains two positive integers n, k, representing, respectively, the number of given integer-coordinate points and the number of integer-coordinate points that may be freely added.

The next n lines describe the points; the i-th line contains two positive integers x_i, y_i, giving the horizontal and vertical coordinates of the i-th given point.

Output

Output one integer: the maximum length of a sequence satisfying the requirements.

Sample 1 input

```
8 2
3 1
3 2
3 3
3 6
1 2
2 2
5 5
5 3
```

Sample 1 output

```
8
```

Sample 2 input

```
4 100
10 10
15 25
20 20
30 30
```

Sample 2 output

```
103
```

Sample 3 input

Provided as the downloadable file `point/point3.in` in the attachment `point.zip`.

Sample 3 output

Provided as the downloadable file `point/point3.ans` in the attachment `point.zip`.

The third sample satisfies $k = 0$.

Sample 4 input

Provided as the downloadable file `point/point4.in` in the attachment `point.zip`.

Sample 4 output

Provided as the downloadable file `point/point4.ans` in the attachment `point.zip`.

Constraints

For all data, it is guaranteed that $1 \leq n \leq 500$ and $0 \leq k \leq 100$. The horizontal and vertical coordinates of every given integer-coordinate point satisfy $1 \leq x_i, y_i \leq 10^9$, and all given points are guaranteed to be distinct. The horizontal and vertical coordinates of the freely added integer-coordinate points are unrestricted.

Test case	$n \leq$	$k \leq$	$x_i, y_i \leq$
$1 \sim 2$	10	0	10
$3 \sim 4$	10	100	100
$5 \sim 7$	500	0	100
$8 \sim 10$	500	0	10^9
$11 \sim 15$	500	100	100
$16 \sim 20$	500	100	10^9

Source

Translated from the following Luogu problem pages, in contest order:

1. `https://www.luogu.com.cn/problem/P8813`

2. https://www.luogu.com.cn/problem/P8814
3. https://www.luogu.com.cn/problem/P8815
4. https://www.luogu.com.cn/problem/P8816

The downloadable sample filenames above are those listed on these pages. The page headers do not display the contest date or sitting hours; the date and hours in this document follow the supplied contest details.